

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE COURSE CODE: CSA1223

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)
Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks:25

ANNEXURE A APPLICATION BASED QUESTIONS

787.8

Code of Conduct:

I B.Poorvaja(192473008) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

B.Poorvaja

ANNEXURE A

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.
2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch-decode-execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.
3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.
5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

MARKS ALLOCATION

Criteria	Understanding of Problem Context (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (20%)	
Q1	3	6	3	3	2	17
Q2	3	5	3	3	2	16
Q3	4	4	4	4	1.5	17.5
Q4	3	4	3	3	1.5	14.5
Q5	2	3	3	3	2	13

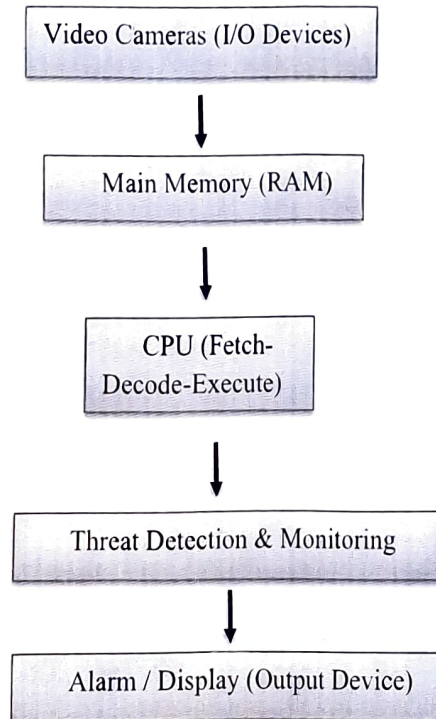
78

RUBRICS FOR CALCULATION:

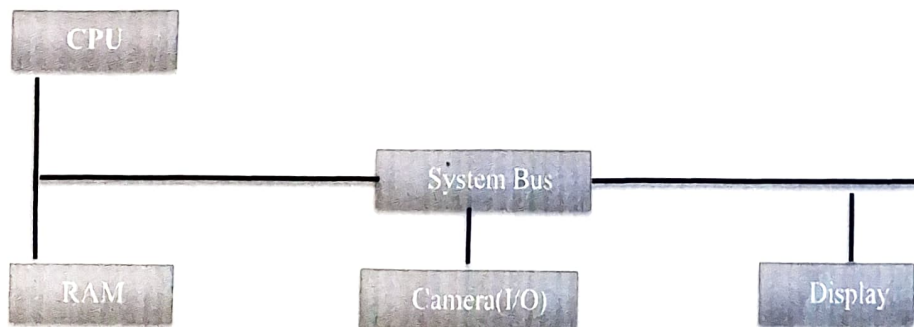
Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements ⁴	Good understanding with minor gaps ³	Basic understanding; some requirements unclear ²	Poor understanding or misinterpretation of the problem ¹
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems ⁶	Applies relevant concepts correctly with minor errors ⁵	Applies basic concepts with limited accuracy ⁴	Fails to apply appropriate concepts ¹⁻²
Problem-Solving Approach	20	Logical, well-structured, and efficient approach ⁴	Mostly logical approach with minor gaps ³	Basic approach with some inconsistencies ²	Illogical or unclear approach ¹
Accuracy of Solution	20	Completely correct solution with proper justification ⁴	Mostly correct with minor errors ³	Partially correct solution ²	Incorrect or incomplete solution ¹
Clarity	10	Clear, well-organized, and properly explained solution ²	Understandable with minor clarity issues ^{1.5}	Limited clarity and explanation ¹	Poorly presented and difficult to understand ⁰

1. Smart City Surveillance System

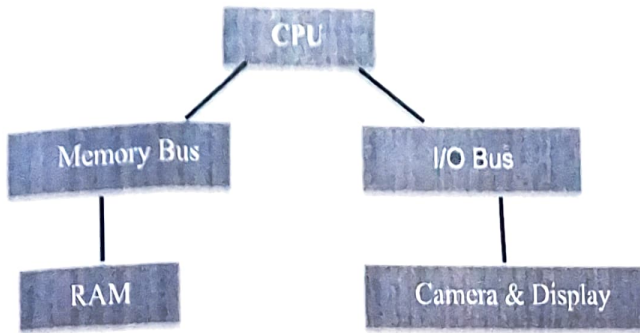
In the given smart city surveillance system, multiple cameras continuously send video streams to the processor for real-time threat detection. During peak traffic hours, the CPU receives a huge amount of data from different I/O devices. Since every video frame must first pass through memory before processing, the communication bus becomes heavily occupied, resulting in delays.



If the system uses a single-bus architecture, the CPU, memory, and I/O devices share one communication path. When many cameras try to transfer data simultaneously, only one transfer can occur at a time. This creates a bottleneck, forcing the CPU to wait for memory access and delaying threat detection.



A better solution is to use a multi-bus architecture. Separate buses allow the CPU, memory, and I/O devices to communicate simultaneously. While one camera transfers data to memory, the CPU can process another frame without waiting. This significantly improves throughput and reduces response time.



The instruction execution cycle also affects performance. During the fetch stage, the CPU retrieves instructions from memory. During decode, it identifies the operation, and during execute, it processes the video data. Frequent memory accesses increase execution time. Using instruction pipelining, cache memory, and DMA (Direct Memory Access) reduces waiting time because frequently used instructions remain in cache and large video transfers bypass the CPU.

Solution:

- Replace the single bus with a multi-bus architecture.
- Use cache memory to reduce memory access time.
- Implement DMA for video transfers.
- Apply pipelining to execute multiple instructions simultaneously.

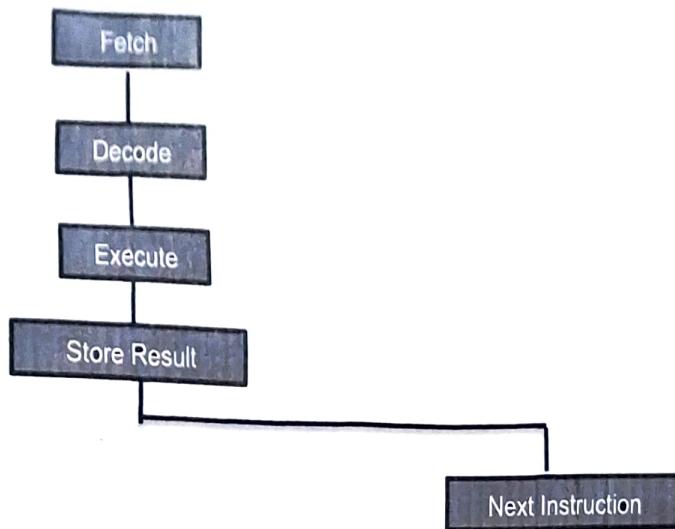
Conclusion:

By improving communication between CPU, memory, and I/O devices and optimizing the instruction execution cycle, the surveillance system can detect threats faster and respond efficiently during peak traffic conditions.

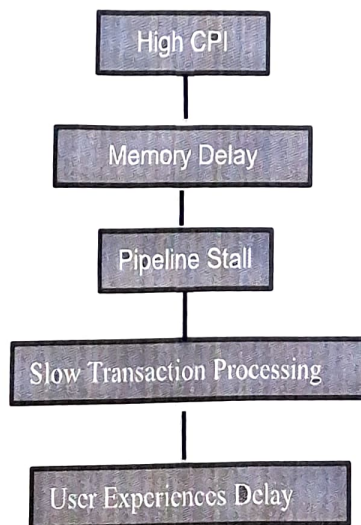
2. Real-Time Stock Trading Application

In the real-time stock trading application processes millions of buy and sell transactions every second. During periods of high trading activity, users experience delays in transaction processing because the processor has a high Cycles Per Instruction (CPI). A high CPI means that the CPU requires more clock cycles to execute each instruction, reducing the number of transactions that can be processed in a given time. Since stock trading systems require instant execution of orders, even a small delay can affect trading accuracy and customer satisfaction.

The performance of the processor depends on the fetch–decode–execute cycle. During the fetch stage, the CPU retrieves instructions from memory. In the decode stage, it identifies the type of operation to be performed. During the execute stage, arithmetic, logical, or branch operations are carried out, and finally, the result is stored back in memory. In this application, frequent memory accesses and branch instructions slow down every stage of this cycle. When the CPU frequently waits for data from main memory, the execution time increases. Similarly, branch instructions interrupt the normal flow of execution because the processor must determine which instruction should be executed next, causing pipeline stalls.



The high CPI can also be caused by cache misses, where the required data is not available in cache memory and must be fetched from the slower main memory. During heavy trading periods, this delay is repeated millions of times, significantly affecting the overall processing speed. As a result, transaction confirmations are delayed, reducing the reliability of the trading platform.



To improve performance, several architectural enhancements can be implemented. Cache memory should be increased so that frequently accessed transaction data and instructions are available closer to the CPU, reducing memory access time. Instruction pipelining allows multiple instructions to be processed simultaneously at different stages of execution, thereby increasing throughput. Branch prediction enables the processor to predict the outcome of branch instructions, minimizing pipeline stalls. Using superscalar or multi-core processors further improves performance by executing multiple instructions or multiple transactions in parallel. Faster RAM and efficient memory hierarchy also reduce memory latency and lower CPI.

Solutions

- Increase cache memory to reduce memory access delays.
- Implement instruction pipelining.
- Use branch prediction techniques.

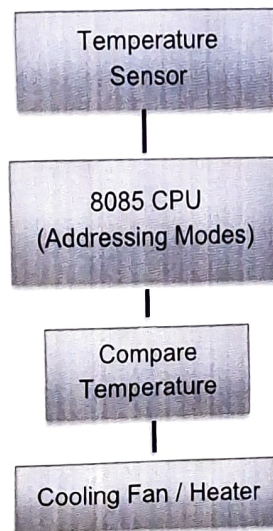
- Upgrade to multi-core or superscalar processors.
- Improve memory hierarchy using faster RAM and cache.

Conclusion

The delays in the stock trading application are mainly caused by a high CPI due to frequent memory accesses and branch instructions. Optimizing the fetch–decode–execute cycle through cache memory, pipelining, branch prediction, and advanced processor architectures significantly reduces execution time. These improvements enable the processor to execute more instructions per second, ensuring faster transaction processing, improved reliability, and better user experience during periods of heavy trading.

3. Industrial Temperature Control System using 8085

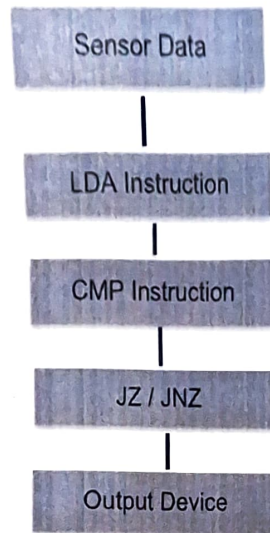
In the given industrial temperature control system, an 8085 microprocessor continuously receives temperature readings from sensors and processes them to control actuators such as cooling fans or heaters. However, the system occasionally processes incorrect temperature values because of the improper use of addressing modes in the program. Since industrial automation requires precise monitoring and control, even a small programming error can result in inaccurate temperature regulation, affecting the safety and efficiency of the entire system.



Addressing modes determine how the processor accesses data required for executing an instruction. If the programmer selects the wrong addressing mode, the processor may fetch incorrect data from memory or registers. For example, if Immediate Addressing is used instead of Direct Addressing, the CPU processes a constant value instead of the actual sensor reading stored in memory. Similarly, an incorrect address stored in the HL register during Register Indirect Addressing may cause the processor to read data from an unintended memory location. These errors lead to incorrect temperature calculations and improper operation of the actuators.

The 8085 instruction set provides several instructions that help in accurate data processing. The LDA instruction reads sensor values from memory, MOV transfers data between registers, CMP compares the measured temperature with the desired value, and JZ or JNZ makes decisions based on the comparison result. Instructions such as IN and OUT are used to

communicate with input sensors and output devices. Proper use of these instructions ensures that the correct sensor data is processed before any control action is taken.



To improve the reliability of the system, the programmer should carefully select the appropriate addressing mode for each operation. Sensor values should always be accessed using the correct memory addresses, and register contents should be verified before execution. Proper validation of input data, regular testing of the program, and careful use of conditional jump instructions further improve system accuracy. Debugging tools can also be used to identify addressing errors before deploying the program in the industrial environment.

Solutions

- Use Direct Addressing for accessing sensor data stored in memory.
- Verify register contents before using Register Indirect Addressing.
- Use LDA, MOV, CMP, JZ, and JNZ instructions correctly.
- Validate sensor inputs before processing.
- Thoroughly test and debug the program before implementation.

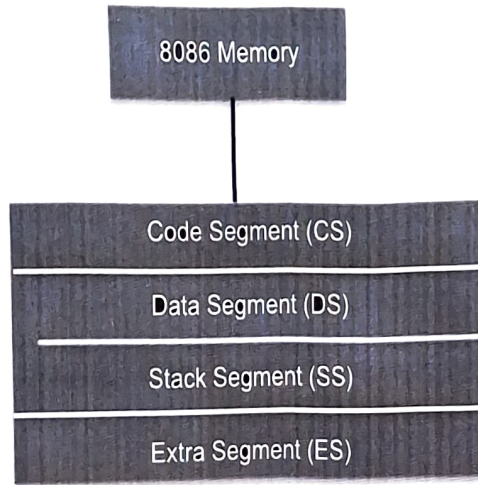
Conclusion

The incorrect temperature readings are mainly caused by improper use of addressing modes in the 8085 program. Selecting the correct addressing mode and effectively using the 8085 instruction set ensures accurate data access, reliable temperature monitoring, and efficient actuator control. This improves the overall safety, accuracy, and performance of the industrial temperature control system.

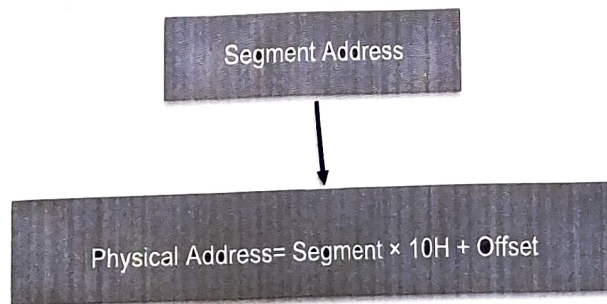
4. 8086 Segmented Memory System

In the given scenario, the computing system uses an 8086 microprocessor to execute multiple programs simultaneously. The processor organizes memory using segmentation, which divides memory into different logical sections. However, the system encounters problems such as incorrect data access and stack overflow because the segment registers are not managed properly. These issues reduce program reliability and may even cause the system to crash.

The 8086 divides memory into four main segments: the Code Segment (CS) stores program instructions, the Data Segment (DS) stores variables and program data, the Stack Segment (SS) stores temporary data, function return addresses, and local variables, while the Extra Segment (ES) provides additional storage mainly for string operations. The processor calculates the physical memory address by combining the segment address with the offset address. Therefore, every instruction depends on the correct initialization of the segment registers.



If the Data Segment (DS) points to an incorrect location, the processor accesses the wrong memory area, resulting in incorrect data being processed. Similarly, if the Stack Segment (SS) is not properly managed, repeated PUSH operations without corresponding POP operations increase the stack size until it exceeds its allocated memory, leading to a stack overflow. This may overwrite important program data or terminate program execution unexpectedly.



Proper instruction execution also depends on the fetch-decode-execute cycle. During execution, the processor must fetch instructions from the correct code segment and access data from the correct data segment. Incorrect segment values interrupt this process and lead to execution errors. Appropriate addressing modes such as Based Addressing, Indexed Addressing, and Based Indexed Addressing should be used carefully to ensure that memory locations are accessed correctly.

To avoid these problems, programmers should initialize all segment registers before execution, maintain proper stack management by balancing every PUSH with a POP instruction, and continuously monitor stack usage during program execution. Proper debugging and testing can identify segmentation errors before deployment.

Solutions

- Initialize CS, DS, SS, and ES correctly before execution.

- Balance every PUSH instruction with a corresponding POP instruction.
- Use appropriate addressing modes for memory access.
- Monitor stack usage to prevent overflow.
- Test segment register values during program execution.

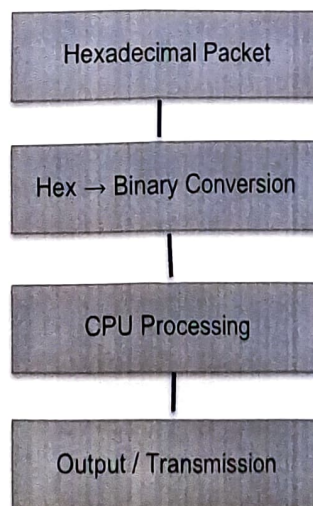
Conclusion

The problems in the 8086 system arise mainly because of improper segment handling and poor stack management. Correct initialization of segment registers, careful use of addressing modes, and efficient management of the instruction execution cycle ensure reliable memory access, prevent stack overflow, and enable multiple programs to execute correctly without errors.

5. Data Communication System

In the given data communication system, packet information is represented in hexadecimal format because it is compact, easy to read, and convenient for debugging. However, computers execute instructions only in binary. In this scenario, the software engineer incorrectly interprets hexadecimal values during conversion, resulting in incorrect packet processing. Since communication systems depend on accurate data transmission, even a small conversion error can affect the entire system.

Each hexadecimal digit represents four binary bits, making hexadecimal an efficient representation of binary data. Before processing a packet, the CPU converts the hexadecimal values into binary. If the conversion is incorrect, the processor interprets the packet differently, leading to incorrect memory addresses, invalid instructions, or corrupted data. For example, if the hexadecimal value 3A is converted incorrectly, the binary representation changes, causing the CPU to process the wrong information. Such errors may result in communication failures, packet loss, or unexpected system behaviour.



These conversion errors also affect the instruction execution cycle. During the fetch stage, the processor retrieves incorrect instructions because of the wrong binary value. During decoding, the CPU misinterprets the operation, and during execution, incorrect data is processed. Consequently, the entire communication process becomes unreliable and slower.

Modern CPU architecture provides several mechanisms to minimize such problems. Cache memory speeds up access to frequently used data, while error detection and correction

mechanisms help identify incorrect values before execution. Efficient processor designs with pipelining and parallel processing improve execution speed and reduce delays. In addition, CPU benchmarking evaluates processor performance using metrics such as CPI, execution time, throughput, clock speed, and MIPS. These measurements help engineers detect performance bottlenecks and optimize the processor for real-time communication systems.

Hex : 3A
3 → 0011
A → 1010
Result
00111010

Solutions

- Verify hexadecimal-to-binary conversions before execution.
- Use automated conversion and validation techniques.
- Implement error detection mechanisms.
- Improve CPU performance using cache memory and pipelining.
- Regularly benchmark the processor to identify performance issues.

Conclusion

The incorrect interpretation of hexadecimal values directly affects binary processing and instruction execution, leading to communication errors and reduced system reliability. Accurate number system conversion, combined with efficient CPU architecture and regular benchmarking, ensures faster packet processing, minimizes errors, and improves the overall performance of real-time data communication systems.